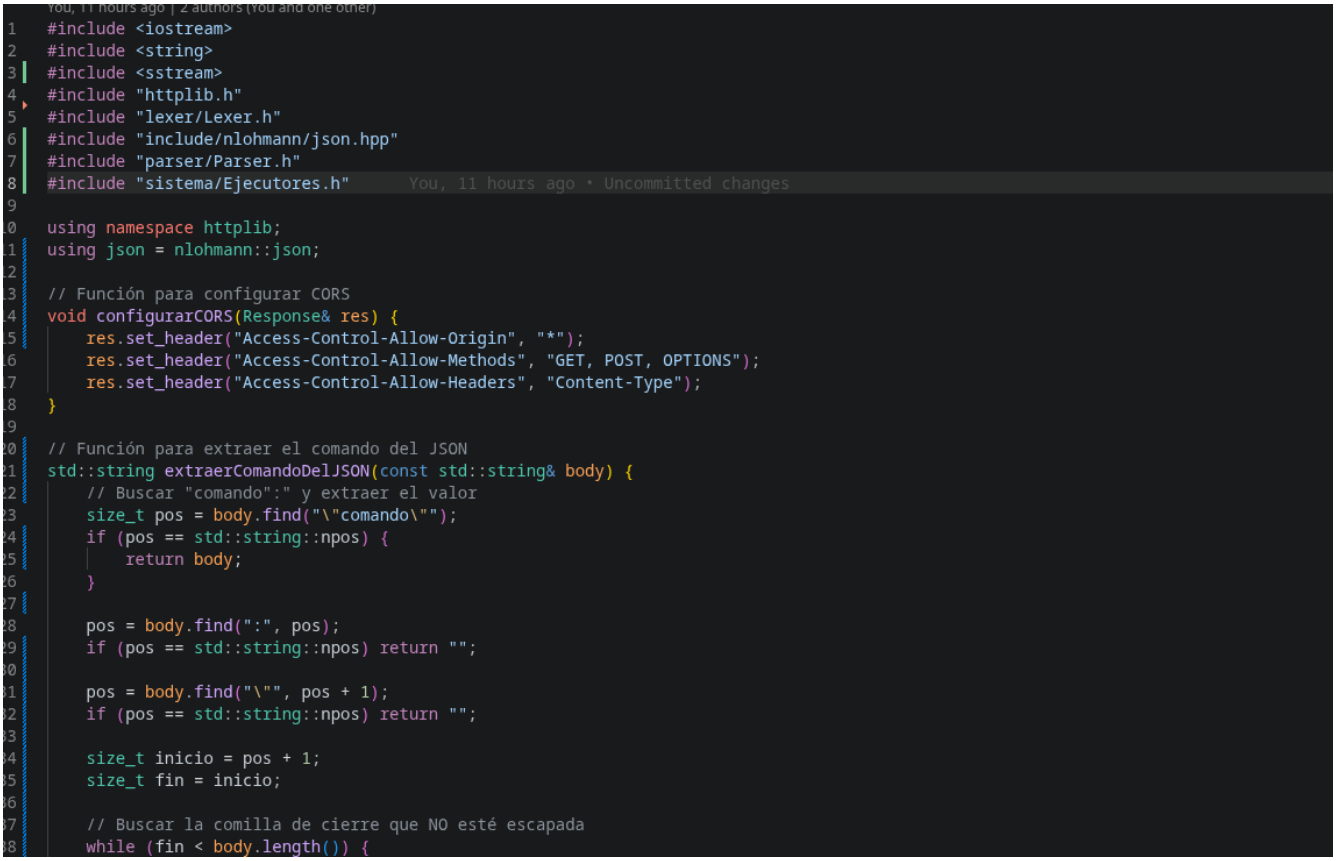
MANUAL TECNICO



```
You, 11 hours ago | 2 authors (You and one other)
#include <iostream>
#include <string>
#include <sstream>
#include "httplib.h"
#include "lexer/Lexer.h"
#include "include/nlohmann/json.hpp"
#include "parser/Parser.h"
#include "sistema/Ejecutores.h"      You, 11 hours ago • Uncommitted changes

using namespace httplib;
using json = nlohmann::json;

// Función para configurar CORS
void configurarCORS(Response& res) {
    res.set_header("Access-Control-Allow-Origin", "*");
    res.set_header("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
    res.set_header("Access-Control-Allow-Headers", "Content-Type");
}

// Función para extraer el comando del JSON
std::string extraerComandoDelJSON(const std::string& body) {
    // Buscar "comando":" y extraer el valor
    size_t pos = body.find("\"comando\"");
    if (pos == std::string::npos) {
        return body;
    }

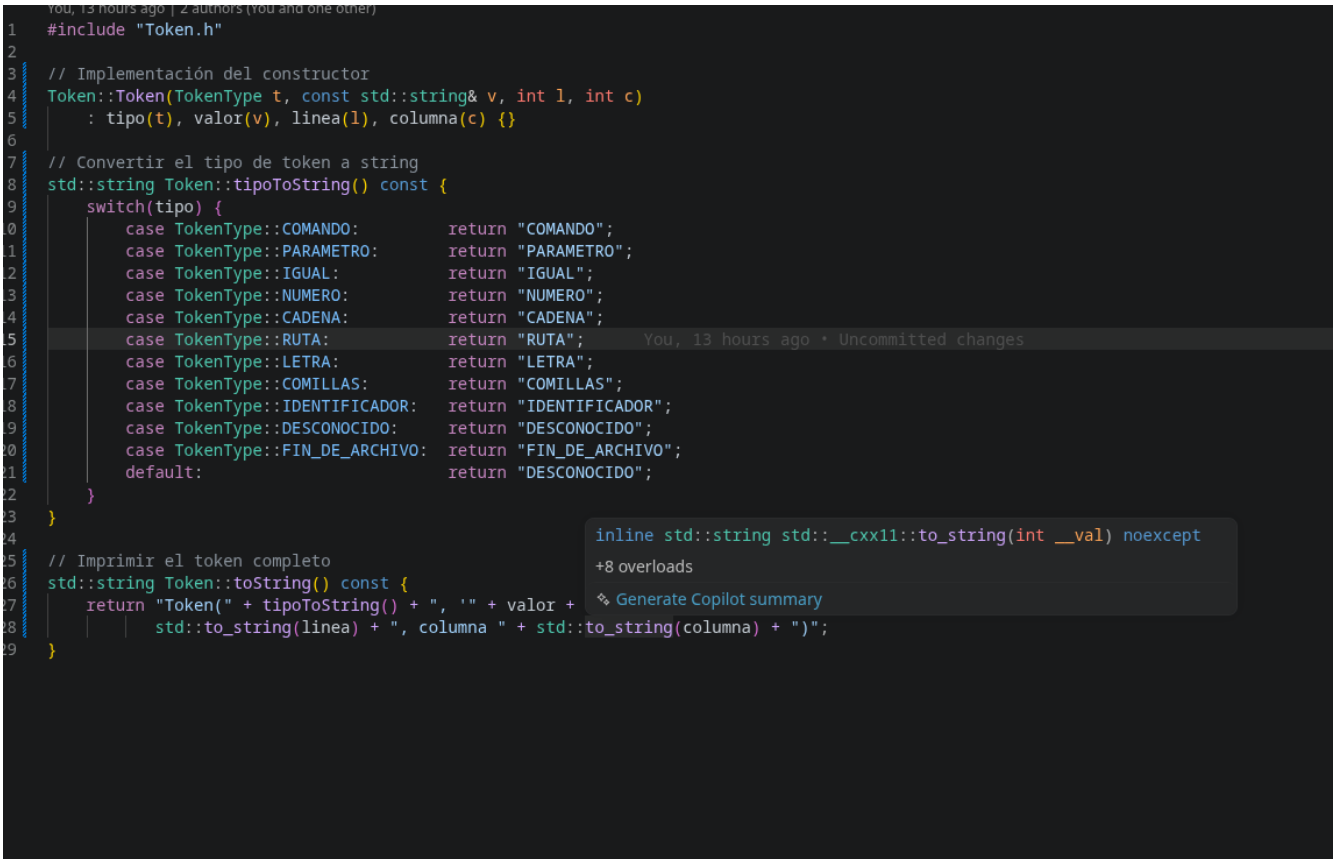
    pos = body.find(":", pos);
    if (pos == std::string::npos) return "";

    pos = body.find("\"", pos + 1);
    if (pos == std::string::npos) return "";

    size_t inicio = pos + 1;
    size_t fin = inicio;

    // Buscar la comilla de cierre que NO esté escapada
    while (fin < body.length()) {
```

la clase main.cpp es la que maneja toda la comunicación entre el frontend y el backend



```
You, 13 hours ago | 2 authors (You and one other)
#include "Token.h"

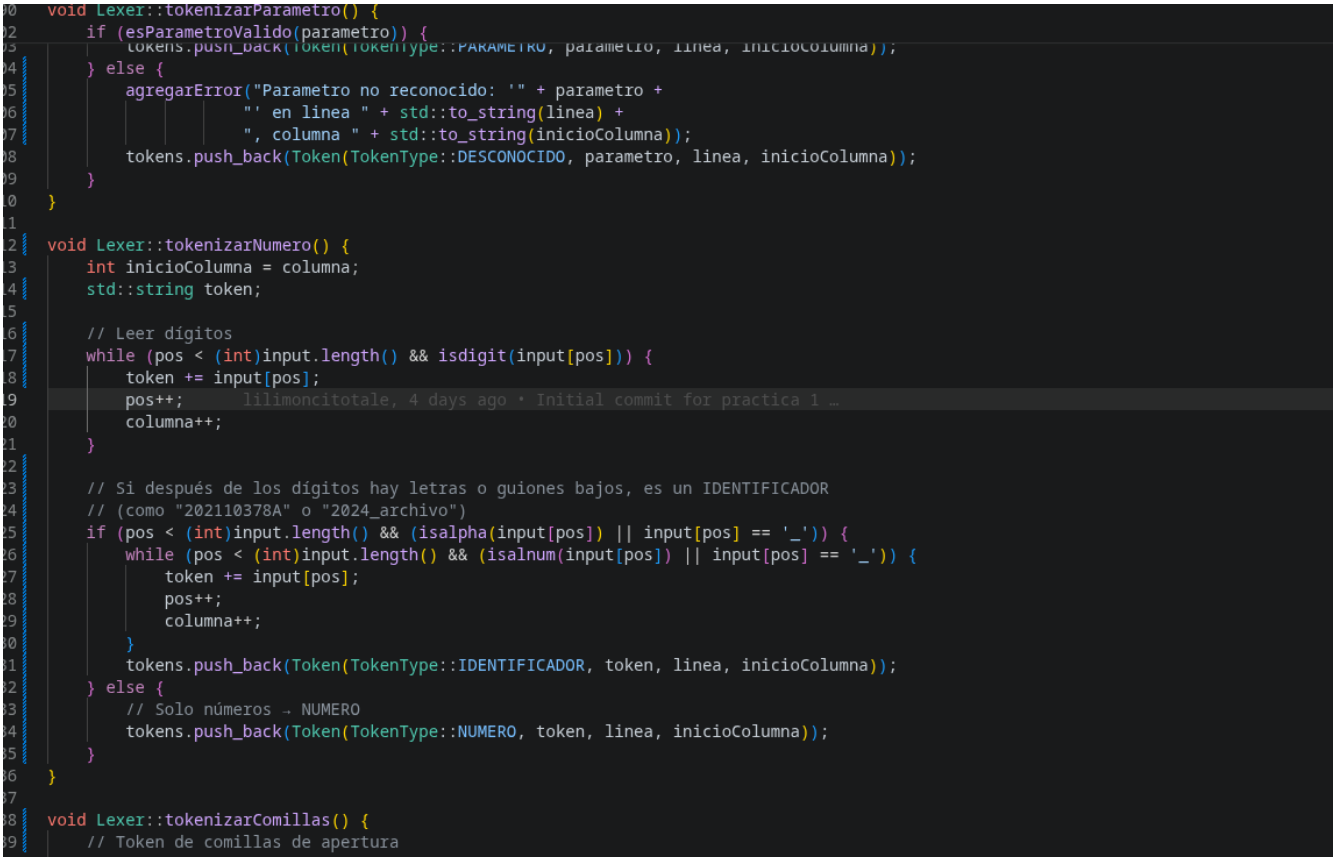
// Implementación del constructor
Token::Token(TokenType t, const std::string& v, int l, int c)
    : tipo(t), valor(v), linea(l), columna(c) {}

// Convertir el tipo de token a string
std::string Token::tipoToString() const {
    switch(tipo) {
        case TokenType::COMANDO:          return "COMANDO";
        case TokenType::PARAMETRO:        return "PARAMETRO";
        case TokenType::IGUAL:            return "IGUAL";
        case TokenType::NUMERO:           return "NUMERO";
        case TokenType::CADENA:           return "CADENA";
        case TokenType::RUTA:             return "RUTA";      You, 13 hours ago • Uncommitted changes
        case TokenType::LETRA:            return "LETRA";
        case TokenType::COMILLAS:         return "COMILLAS";
        case TokenType::IDENTIFICADOR:    return "IDENTIFICADOR";
        case TokenType::DESCONOCIDO:      return "DESCONOCIDO";
        case TokenType::FIN_DE_ARCHIVO:   return "FIN_DE_ARCHIVO";
        default:                          return "DESCONOCIDO";
    }
}

// Imprimir el token completo
std::string Token::toString() const {
    return "Token(" + tipoToString() + ", '" + valor +
           std::to_string(linea) + ", columna " + std::to_string(columna) + ")";
}
```

```
inline std::string std::__cxx11::to_string(int __val) noexcept
+8 overloads
Generate Copilot summary
```

la clase token solo se encarga de definir los tokens que se usaran para en analisis del texto



```
void Lexer::tokenizarParametro() {
    if (esParametroValido(parametro)) {
        tokens.push_back(Token(TokenType::PARAMETRO, parametro, linea, inicioColumna));
    } else {
        agregarError("Parametro no reconocido: '" + parametro +
                     "' en linea " + std::to_string(linea) +
                     ", columna " + std::to_string(inicioColumna));
        tokens.push_back(Token(TokenType::DESCONOCIDO, parametro, linea, inicioColumna));
    }
}

void Lexer::tokenizarNumero() {
    int inicioColumna = columna;
    std::string token;

    // Leer dígitos
    while (pos < (int)input.length() && isdigit(input[pos])) {
        token += input[pos];
        pos++;      lilimoncitotale, 4 days ago • Initial commit for practica 1 …
        columna++;
    }

    // Si después de los dígitos hay letras o guiones bajos, es un IDENTIFICADOR
    // (como "202110378A" o "2024_archivo")
    if (pos < (int)input.length() && (isalpha(input[pos]) || input[pos] == '_')) {
        while (pos < (int)input.length() && (isalnum(input[pos]) || input[pos] == '_')) {
            token += input[pos];
            pos++;
            columna++;
        }
        tokens.push_back(Token(TokenType::IDENTIFICADOR, token, linea, inicioColumna));
    } else {
        // Solo números → NUMERO
        tokens.push_back(Token(TokenType::NUMERO, token, linea, inicioColumna));
    }
}

void Lexer::tokenizarComillas() {
    // Token de comillas de apertura
```

la clase lexer es la que se encarga de ir verificando y asignando los tipos a los tokens y formando los parametros

```
942
943             if (pos >= (int)tokens.size()) {
944                 errores.push_back("Error: Se esperaba un valor para -id");
945                 return false;
946             }
947
948             if (tokens[pos].tipo == TokenType::IDENTIFICADOR ||
949                 tokens[pos].tipo == TokenType::LETRA ||
950                 tokens[pos].tipo == TokenType::CADENA) {
951                 parametros.id = tokens[pos].valor;
952                 parametros.id_presente = true;
953                 pos++;
954             } else {
955                 errores.push_back("Error: Se esperaba un valor para -id, pero se encontro: " + tokens[pos].valor);
956                 return false;
957             }
958         }
959         else if (token.valor == "-path"){
960             if (parametros.path_presente){
961                 errores.push_back("Error: -path repetido");
962                 return false;
963             }
964             pos++;
965
966             if (!esperarToken(TokenType::IGUAL, "Se esperaba '=' despues de -path"))return false;
967
968             if (pos >= (int)tokens.size()){
969                 errores.push_back("Error: Se esperaba una ruta para -path");
970                 return false;
971             }
972
973             if (tokens[pos].tipo == TokenType::RUTA || tokens[pos].tipo == TokenType::CADENA){
974                 parametros.path = tokens[pos].valor;
975                 parametros.path_presente = true;
976                 path_encontrado = true;
977                 pos++;
978
979             }else {
980                 errores.push_back("Error: Se esperaba una ruta para -path, pero se encontro: " + tokens[pos].valor);
```

la clase parser se encarga de verificar que las estructuras de los comandos sea la correcta para irlos ejecutando posteriormente

```cpp
std::string ejecutarRMUSR(const ParametrosComando& params) {
    if (sistema.eliminarUsuario(params.id, params.user)) {
        return " Usuario eliminado exitosamente: " + params.user;
    } else {
        return " Error: El usuario " + params.user + " no existe";
    }
}

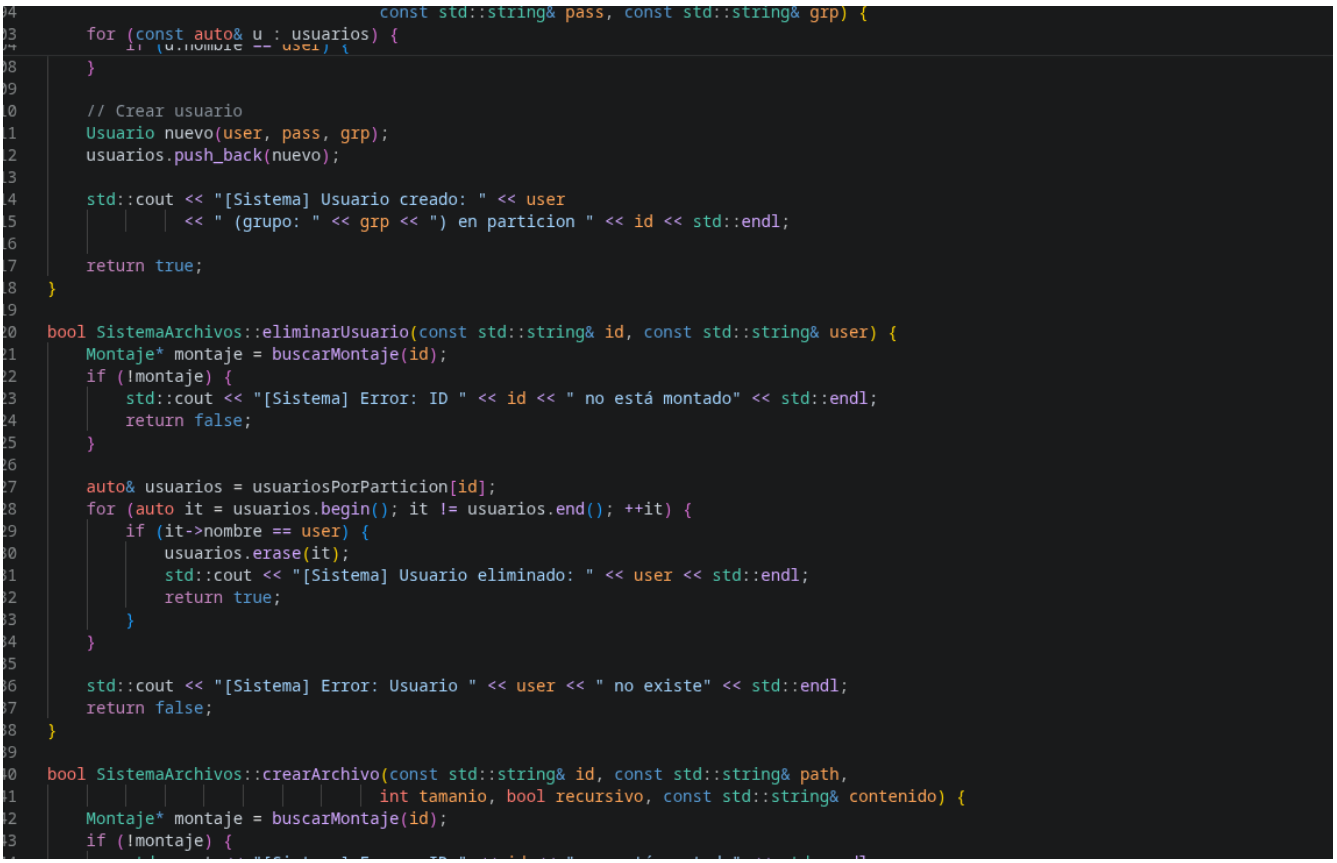
std::string ejecutarMKFILE(const ParametrosComando& params) {
    // Verificar que el ID exista
    Montaje* montaje = sistema.buscarMontaje(params.id);
    if (!montaje) {
        return "Error: El ID " + params.id + " no está montado";
    }

    // Verificar que el archivo no exista
    if (sistema.archivoExiste(params.id, params.path)) {
        return "Error: El archivo " + params.path + " ya existe";
    }

    // Obtener parámetros
    int tamanio = params.size_presente ? std::stoi(params.size) : 0;
    std::string contenido = params.cont_presente ? params.cont : "";

    // Crear el archivo
    if (sistema.crearArchivo(params.id, params.path, tamanio, params.r_flag, contenido)) {
        std::ostringstream msg;
        msg << " Archivo creado exitosamente:\n";
        msg << "   Ruta: " << params.path << "\n";
        msg << "  Tamaño: " << tamanio << " bytes\n";
        msg << "   En partición: " << params.id;
        if (params.r_flag) {
            msg << "\n   Se crearon las carpetas recursivamente";
        }
        return msg.str();
    } else {
        return " Error al crear el archivo";
    }
}
```

la clase ejecutores se encarga de realizar las acciones indicadas en el codifo ingresado por el usuario



```
                                const std::string& pass, const std::string& grp) {
    for (const auto& u : usuarios) {
        if (u.nombre == user) {
    }

    // Crear usuario
    Usuario nuevo(user, pass, grp);
    usuarios.push_back(nuevo);

    std::cout << "[Sistema] Usuario creado: " << user
              << " (grupo: " << grp << ") en particion " << id << std::endl;

    return true;
}

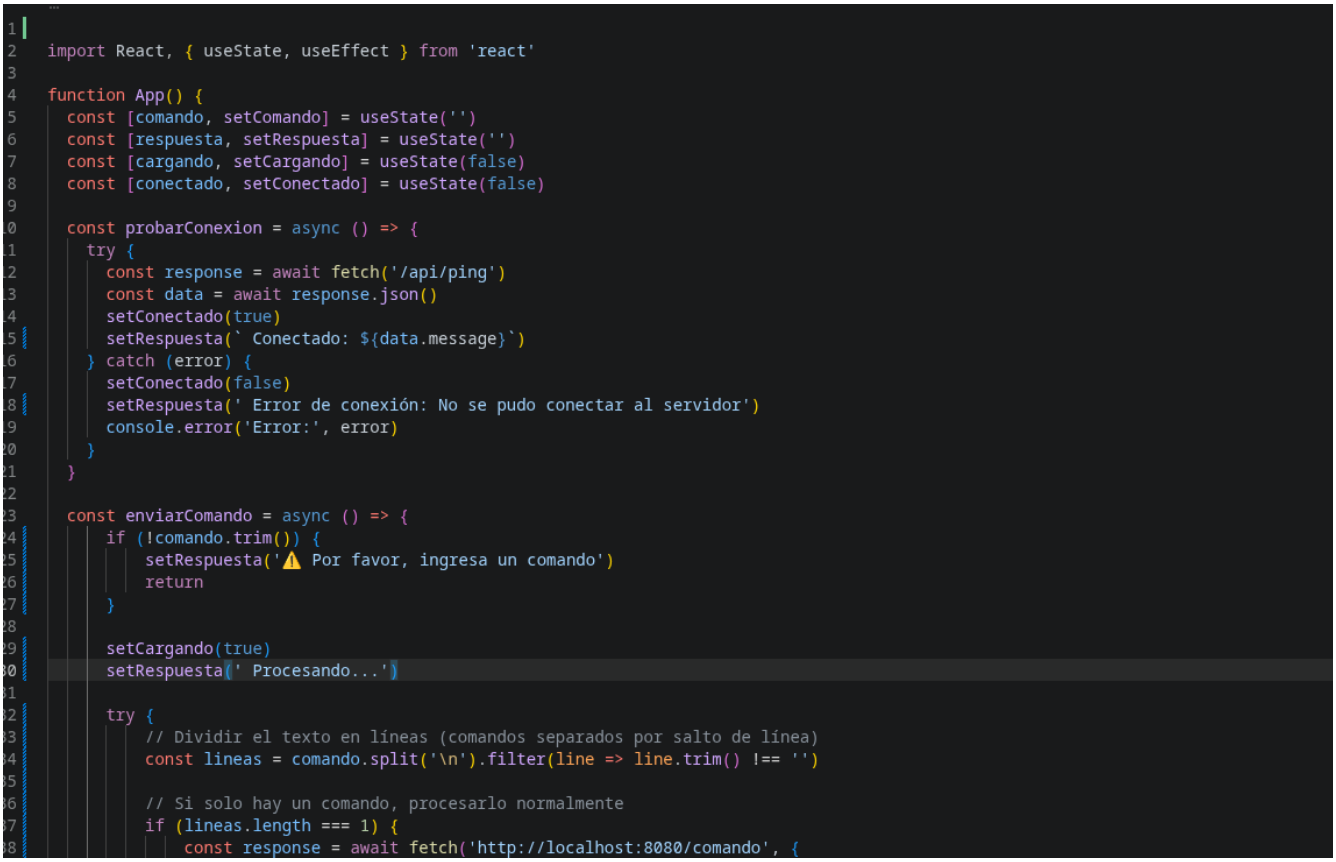
bool SistemaArchivos::eliminarUsuario(const std::string& id, const std::string& user) {
    Montaje* montaje = buscarMontaje(id);
    if (!montaje) {
        std::cout << "[Sistema] Error: ID " << id << " no está montado" << std::endl;
        return false;
    }

    auto& usuarios = usuariosPorParticion[id];
    for (auto it = usuarios.begin(); it != usuarios.end(); ++it) {
        if (it->nombre == user) {
            usuarios.erase(it);
            std::cout << "[Sistema] Usuario eliminado: " << user << std::endl;
            return true;
        }
    }

    std::cout << "[Sistema] Error: Usuario " << user << " no existe" << std::endl;
    return false;
}

bool SistemaArchivos::crearArchivo(const std::string& id, const std::string& path,
                                   int tamanio, bool recursivo, const std::string& contenido) {
    Montaje* montaje = buscarMontaje(id);
    if (!montaje) {
```

la clase sistema de archivos se encarga de simular el funcionamiento de los discos como en este caso no se realiza nada fisico en realidad esta se encarga de simularlo



```jsx
import React, { useState, useEffect } from 'react'

function App() {
  const [comando, setComando] = useState('')
  const [respuesta, setRespuesta] = useState('')
  const [cargando, setCargando] = useState(false)
  const [conectado, setConectado] = useState(false)

  const probarConexion = async () => {
    try {
      const response = await fetch('/api/ping')
      const data = await response.json()
      setConectado(true)
      setRespuesta(` Conectado: ${data.message}`)
    } catch (error) {
      setConectado(false)
      setRespuesta(' Error de conexión: No se pudo conectar al servidor')
      console.error('Error:', error)
    }
  }

  const enviarComando = async () => {
      if (!comando.trim()) {
          setRespuesta('⚠️ Por favor, ingresa un comando')
          return
      }

      setCargando(true)
      setRespuesta(' Procesando...')

      try {
          // Dividir el texto en líneas (comandos separados por salto de línea)
          const lineas = comando.split('\n').filter(line => line.trim() !== '')

          // Si solo hay un comando, procesarlo normalmente
          if (lineas.length === 1) {
              const response = await fetch('http://localhost:8080/comando', {
```

la clase app.jsx se encarga de enviar los datos al back para poder realizar el analisis